

Aiguillage IA

Quand les modèles décident au lieu d'écrire

Jeudi 8 octobre 2026 · [Anne-Marie Roy](#) · Code Academy 123



D'abord, un effet.



L'explication viendra ensuite.

[Chromium Extension - Anti-Pub](#)

Le constat

- ▶ « Classe cet email. » → vous réveillez un modèle de plusieurs centaines de milliards de paramètres.
- ▶ Il **écrit** une réponse... que vous re-parsez ensuite en JSON.
- ▶ Coût et latence d'un modèle génératif, avec une sortie qu'il faut contraindre et valider.

On utilise un moteur d'écriture pour une tâche de décision.

Pourquoi générer du texte quand il faut décider ?

- ▶ Un LLM génère **token par token** : chaque mot dépend des précédents.
- ▶ Même une sortie JSON courte reste une **génération séquentielle**.
- ▶ Conséquence : coût et latence de génération, avec une sortie qu'il faut contraindre et valider.
- ▶ “Si les modèles sont déjà extraordinairement bons pour converser depuis plusieurs années, pourquoi ne voit-on pas beaucoup plus d'automatisation fiable ” — Diogo Almeida, fondateur de TypeSafe
- ▶ L'idée de TypeSafe c'est qu'une partie du problème vient précisément de là : on essaie d'automatiser des décisions avec des modèles conçus avant tout pour générer du langage.

Deux ans de travail discret pour répondre à cette question.

System 1 vs System 2

Kahneman — Thinking, Fast and Slow

System 1

Rapide, automatique, peu coûteux. Il prend une décision à partir de signaux connus, sans dérouler un long raisonnement.

Ex. : reconnaître, classer, choisir, détecter.

System 2

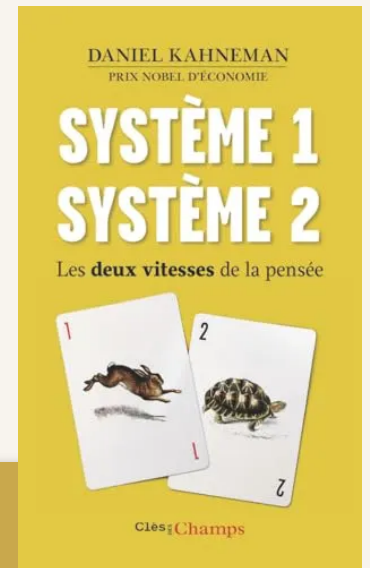
Lent, délibéré, plus coûteux. Il analyse plusieurs éléments, raisonne étape par étape et produit une réponse élaborée.

Ex.: analyser → raisonner → expliquer

Le LLM actuel, c'est un **System 2** qui écrit.

Or 95 % des décisions logicielles sont des System 1 : router, classer, trier, décider oui/non.

Il manquait un modèle System 1 conçu pour le logiciel, pas pour le chat.



TypeSafe AI · Jev

Le pari d'un co-inventeur du RLHF

“We build prod not God” — Une IA conçue pour être intégrée dans des logiciels et automatiser des décisions.
“AI for Software” - “Goal is automation”. — De l’IA pour le logiciel, avec un objectif : l’automatisation.

TypeSafe AI, en bref

2024

Fondation à San Francisco — 2 ans en stealth, sous les radars.

\$40M

Levés, menés par DCVC, fonds de capital-risque spécialisé dans le “deep tech”.

15/09/2026

Sortie publique de Jev

Fondateur : **Diogo Almeida** — équipe technique d'OpenAI, co-inventeur du RLHF (InstructGPT). Avant : Google. Avec lui : Erik Gafni et Sasha Sheng.

L'origine : InstructGPT capte vite ~50 % du marché des LLM mais la valeur part dans le copywriting (Jasper, Copy.ai) : on apprend aux modèles à écrire pour un humain, pas à décider pour du code.

Sa question : « ***Si l'IA devient une API, qui va l'appeler — un humain, ou du code ?*** » Jusqu'ici, on a surtout optimisé l'IA pour parler aux humains — beaucoup moins pour parler directement aux logiciels. Donc, si demain celui qui appelle le service n'est plus un humain, mais un autre programme ou un agent IA, alors les besoins changent.

Son motto : Les humains parlent en langage naturel. Les logiciels parlent en types.

Le chat a progressé pendant dix ans. L'automatisation, pas.

Jev — premier « System One model »

- ▶ Nom emprunté à l'économiste **William Stanley Jevons**.
- ▶ Ne génère **aucun texte** : il rend des **décisions typées**.
- ▶ C'est un “smart if-statements” (un if sémantique) pour le code
- ▶ Évalue plusieurs questions en **un seul passage parallèle**.
- ▶ Tarif : **\$0,042 / M tokens en entrée, sortie gratuite**.
- ▶ Confiance calibrée fournie avec chaque réponse.

Le **paradoxe de Jevons** : quand une technologie devient plus efficace et moins coûteuse, on peut finir par l'utiliser beaucoup plus.
En résumé : **Plus efficace → moins cher → plus d'usages → potentiellement plus de consommation totale.**

Un modèle fait pour être appelé par du code — pas pour discuter avec un humain.

LLM classique vs Jev

Le cœur du sujet

	LLM classique	Jev
Espace	Ouvert (texte libre)	Borné (options connues)
Sortie	Texte libre	Décisions typées
Génération	Token par token	Parallèle, un seul passage
Confiance	Souvent surconfiance	Calibrée, systématique
Idéal pour	Rédiger, résumer, coder	Router, classer, juger

LLM : il génère quand l'espace des réponses est ouvert. Jev : il évalue quand il est borné.

Trois primitives — tout est là

Choice

Choisir parmi des options connues. Ex. :
l'équipe visée → ingénierie / facturation / ventes.

Score

Situer sur une échelle ordonnée. Ex. : niveau
de risque → bas / moyen / haut.

Noul

Évaluer un vrai/faux → renvoie P(vrai). Ex. :
« cet incident est-il urgent ? »

```
{  
  "state": "<texte ou JSON>",  
  "questions": {  
    "queue": { "type": "choice" },  
    "urgent": { "type": "noul" },  
    "frustration": { "type": "score" }  
  }  
}
```

On utilise une ou plusieurs primitives selon le besoin.

Vous déclarez les questions et leurs types ; le modèle rend des probabilités.

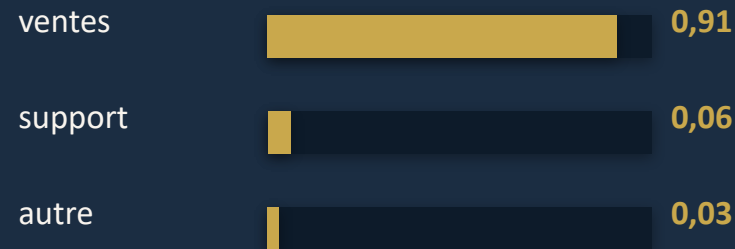
Une décision, en clair

Dix secondes pour voir toute l'architecture

ÉTAT ENTRANT

« Mon compte Stripe ne se connecte plus depuis 3 jours, je perds des ventes. »

CHOICE — QUELLE ÉQUIPE ?



LE CODE DÉCIDE

$p > 0,90 \rightarrow$ routage auto

$0,60 < p \leq 0,90 \rightarrow$ second avis

$p \leq 0,60 \rightarrow$ un humain tranche

Le seuil est une décision d'ingénierie — proportionnée au coût d'une erreur.

Jev estime. Le code décide.

En résumé : Langage naturel $\rightarrow$ jugement structuré $\rightarrow$ probabilité $\rightarrow$ règle métier $\rightarrow$ action.

Typé + calibré : le code prend la décision. Jev fournit le jugement.

La probabilité compte autant que la réponse

- ▶ **91 % / 9 %** → routage automatique raisonnable.
- ▶ **52 % / 48 %** → escalade : plus de contexte, second modèle, ou revue humaine.
- ▶ **Le code possède les seuils et les conséquences.**
- ▶ Un seuil par **action**, ajusté au risque : plus l'erreur coûte cher, plus on exige de confiance.
- ▶ Jev ne décide pas de la conséquence. **Il fournit un jugement sémantique avec une probabilité.** Ensuite, c'est le code métier qui dit quoi faire avec cette incertitude.

Jev extrait le jugement sémantique. Le code décide quoi en faire

RLHF · RLVR · RLCD (Les méthodes d'entraînement)

Le changement est dans l'entraînement

	RLHF	RLVR	RLCD
Récompense	Préférence humaine	Vérification programmée	Probabilités calibrées
Optimise pour	Ce qui plaît à un humain	Ce qui est vérifiable	Une probabilité honnête
Sortie typique	Texte convaincant	Réponse vérifiable	Décision + confiance
Exemples	ChatGPT & co	Maths, code	Jev

RLHF : Reinforcement Learning from Human Feedback

RLVR : Reinforcement Learning with Verifiable Rewards

RLCD : Reinforcement Learning for Calibrated Decisions

Même socle d'apprentissage par renforcement. Objectif entièrement différent.

« Calibré », concrètement

La promesse

Sur un grand nombre de réponses auxquelles Jev attribue ~70 % de confiance, ~70 % devraient être correctes. Comme une prévision météo bien calibrée.

Pourquoi ça change tout

La confiance devient actionnable : $\geq 0,9 \rightarrow$ j'agis seul ; $< 0,6 \rightarrow$ j'escalade vers un humain.

Une confiance qui veut dire quelque chose, c'est ce qui rend l'automatisation possible.

Ce que ça débloque - Cas d'usage concrets

Routeur de coût

Jev regarde la requête et décide si elle peut être traitée par un petit modèle, ou si elle mérite d'être envoyée vers un modèle plus puissant et plus cher. On évite donc d'utiliser le "gros modèle" systématiquement.

Passerelle anti-coût (Évitement du modèle cher)

Si 90 % des cas sont simples, Jev peut les résoudre ou les aiguiller sans jamais appeler un modèle génératif coûteux. Le gros modèle devient l'exception, pas la règle.

Garde-fou d'agent

Jev peut observer ce qu'un agent ou un LLM est en train de faire et juger si une action paraît conforme, risquée ou suspecte. Là encore, il ne prend pas forcément l'action lui-même : il fournit un signal que le code utilise pour autoriser, bloquer ou escalader.

UI réactive

Comme Jev peut répondre très vite, on peut l'utiliser directement dans une interface : à chaque clic ou interaction, une décision sémantique peut être prise sans que l'utilisateur ressente une latence importante.

Anecdote : OpenRouter propose désormais Jev Router comme modèle à part entière : si on le sélectionne, Jev devient l'aiguilleur et choisit lui-même quel LLM doit traiter chaque requête.

Le code décide de la conséquence ; Jev fournit le jugement.

Les chiffres

Chiffres annoncés par TypeSafe

70–500 ms

Latence par appel — un LLM frontalier met 3 à 329 s

\$0,042 / M

Tokens en entrée — sortie gratuite

0 erreur de format/type

Sortie toujours conforme au schéma attendu

40–200×

Plus rapide sur une question « System One »

2 ordres

De grandeur — plus efficace, selon TypeSafe pour des tâches de type System One.

≈ 68 %

Précision annoncée — proche d'un LLM moyen mais avec beaucoup moins de coût et latence.

Moins cher qu'un LLM pour décider. Mais pas pour écrire — et ce sont leurs chiffres.

Cas réel — prédire les réservations d'un standard téléphonique

Sully AI × JEV · 2 029 appels de réceptionniste IA · zéro-shot

2 029

appels réels analysés

38 012

prévisions tour-par-tour
(118 ms de médiane)

AUC 0,78

dès la mi-appel :
« va réserver » vs non

94 %

de ranking correct
en fin d'appel

Source: Sully.ai est une plateforme d'IA spécialisée dans la santé.

Sur 2 029 appels réels, Jev essaie de prédire en temps réel : “ce client va-t-il réserver ?” Sans audio ni transcript, uniquement à partir des traces structurées du logiciel.

Dès la moitié de l'appel, l'AUC atteint 0,78 ; il commence déjà à bien séparer les appels qui vont aboutir à une réservation de ceux qui n'aboutiront pas.

En fin d'appel, le ranking est correct dans 94 % des cas, avec 118 ms de latence médiane, pour un cout total du run de 3\$.

L'intérêt n'est pas seulement de prédire le résultat final : c'est de détecter assez tôt qu'un appel déraile pour pouvoir intervenir.

Source : Sully AI (@muratcan), 29/09/2026 — chiffres rapportés par l'équipe. Caveat assumé : « Jev peut interpréter comme très négatif un incident momentané dans l'appel, alors que l'agent Sully réussit ensuite à récupérer la situation. ».

Décider, pas écrire : l'appel est jugé en vol, la marge d'erreur est explicite — et l'humain reste dans la boucle.

Ce que Jev ne fera PAS

- Rédaction, résumé, génération de code, explications écrites.
- Arithmétique, et tout raisonnement multi-étapes *dépendant*.
- Pas de raisonnement multi-étapes explicite ni de chaîne de pensée — **c'est volontaire**.
- Jev respecte toujours les options définies dans le schéma. **Mais une réponse conforme n'est pas nécessairement une réponse juste.**
- Questions indépendantes : plusieurs décisions dans un même appel à Jev.
 - Ex: Questions indépendantes :
 - “Le message est-il urgent ?”, “Quel est le niveau de frustration ?”, “Quelle est la nature de la demande ?”
 - — Traitement en parallèle
- Questions dépendantes : le code utilise une première décision pour déterminer l'étape suivante
 - Ex: Question dépendante : “ Si la demande est un remboursement, faut-il l'escalader ?” — Traitement séquentiel via le code

Le type-safety garantit la forme. Pas le jugement.

Construire avec Jev

« *It's like ML without the ML.* » — Diogo Almeida

Découper en petites décisions

Ne demandez pas « faut-il refuser ? ». Posez dix questions indépendantes et combinez. Chaque raté devient un test — pas un prompt qu'on rallonge.

De l'état structuré, pas des gros strings

JSON pour l'état, les critères et les instructions.

« Un system message, c'est une variable globale dégueulasse. » Diogo

La confiance pilote l'escalade

Un seuil par action, proportionné au coût d'une erreur : au-dessus on agit seul, en dessous on passe la main à un humain. Le modèle fournit un jugement et une probabilité.

Ça s'insère, ça ne remplace pas

Un appel HTTP, un nœud n8n. Le reste du code ne bouge pas — et ça se teste comme n'importe quel composant.

On l'appelle comme un service.

Décomposer, mesurer, corriger chirurgicalement : le ML redevient de l'ingénierie.

Le vrai sujet

Au-delà de Jev

- Jev n'est pas important parce qu'il serait « *meilleur* » qu'un LLM généraliste. Il ne l'est pas — il fait autre chose.
- Il remet en cause une idée reçue : que **tout** problème d'IA doive se résoudre avec un LLM génératif.
- Or tous les problèmes ne sont pas des problèmes de génération.
- Si l'espace de sortie est **ouvert** — écrire, résumer, expliquer, créer du code — un LLM génératif est naturel.
- Si l'espace est **borné** — choisir entre quelques options, attribuer un score, décider vrai ou faux, router une demande — alors un modèle spécialisé pour la décision peut être beaucoup plus approprié.
- Le réflexe à garder après ce soir : « **quel est le bon outil pour cette tâche ?** » Parfois ce sera un grand LLM. Parfois un petit modèle local. Parfois du code déterministe. Parfois Jev.

Le bon modèle pour la tâche. C'est tout — et c'est déjà beaucoup.

Aller plus loin

- typesafe.ai · docs.typesafe.ai — le produit et la doc.
- Blog fondateur : « [Introducing System One Models & Jev](#) ».
- Catalogue de cas d'usage : github.com/Anil-matcha/awesome-jev-by-typesafe
- Notre fork OpenRouter : github.com/royam0820/jev-adblock-openrouter
- Démo kit : <https://github.com/royam0820/Jev-Meetup>

Tout ce qui a été montré ce soir est dans le kit que je vous partage.

Maintenant, en direct.

JEV Démonos



Jev - Chrome Extension anti-pub

Decision Router - LLM local vs Cloud Dev

N8n - triage emails

N8n - Evaluation - LLM judge vs Jev

Google Sheets - Jev

Le fameux exemple du carwash ! (Jev state mal défini vs Jev state mieux défini)

Une application amusante via Jev

Laya / Ollaya (jev opensource)

Jev - Suppression Encarts Publicitaire - Chrome Extension

Openrouter : [typesafe/jev-1.13](https://openrouter.ai/typesafe/jev-1.13)

Réf.: [Jev adblock openrouter](#)

Fichier	Rôle
src/content.js	Le cœur du traitement DOM. Il s'exécute directement dans la page web. Il détecte les encarts potentiels, les décrit, reçoit le score de l'IA, et supprime physiquement les éléments HTML .
src/background.js	Le service worker en arrière-plan. Il fait le pont entre le content.js et l'API IA (charge la clé API du stockage Chrome, gère le compteur du badge d'extension).
src/typesafe.js	Le module d'intégration avec l'API IA. Il prépare le payload JSON avec les questions de jugement sémantique et extrait les probabilités d'ad.

Cinq Etapes :

- Repérage des encarts candidats par regex et sélecteurs de mots clés (src/content.js)
- Transformation en méta-données JSON (src/content.js)
- Jugement par l'IA (src/typesafe.js & src/background.js)
- Suppression effective du DOM (src/content.js)
- Observation dynamique (Lazy Loading / Scroll) (src/content.js). Pour supprimer les pubs qui s'affichent au défilement ou de façon dynamique

Aiguillage IA — même cas, deux moteurs

LOCAL = Qwen3-1.7B · \$0 · hors-ligne | CLOUD = JEV TypeSafe · ≈ 0,000016 \$/décision | seuil AUTO = 0,70

Réf.: [Decision Router](#)

Cas	JEV cloud	LOCAL · Qwen (\$0)
Email urgent	client urgent · 1.00 · 399 ms ✓	client urgent · 1.00 · 7 455 ms ✓
Routage social	facebook · 0.83 · 305 ms ✓	instagram · 1.00 · 2 690 ms ⚠
Cas ambigu	instagram · 0.52 · 254 ms → revue humaine	instagram · 1.00 · 2 690 ms ✓

Format : réponse · confiance · latence — seuil 0,70 : au-dessus, la machine tranche ; en dessous, un humain reprend la main.

≈ 320 ms

JEV — moyenne par décision

≈ 4 278 ms

LOCAL — moyenne par décision

Le local tranche. JEV sait aussi quand ne pas trancher.

n8n — trier les emails, sans les lire

Gmail → JEV (typesafe/jev-1.13) → l'étiquette se pose toute seule · 1 appel HTTP par email · 0 texte généré · seuil 0,70



Confiance $\geq 0,70$ → étiquette posée automatiquement

Facturation

Sécurité

Newsletter

Perso

Spam_JEV

Cinq dossiers, aucune lecture humaine : l'email est rangé à l'arrivée.

Confiance
< 0,70

→ À revoir (humain)

*le modèle hésite :
c'est un humain qui tranche.*

6

étiquettes en jeu
(5 auto + 1 humain)

~300 ms

par email (JEV)

≈ 0,00002 \$

par email trié

≤ 1 min

délai de relève

Aucun texte généré : une décision, une étiquette. Sous le seuil, l'email attend un humain.

n8n - LLM - Jev Judge - Ce que JEV renvoie : les 3 nombres

probabilities · choice · confidence

1

probabilities

La distribution sur les classes. C'est ici que se lit « à quel point c'est correct ».

correct 0.52 · partiel 0.48 · incorrect 0.00

2

choice

La classe gagnante. Une décision typée, directement exploitable dans le workflow.

→ correct

3

confidence

La sûreté du modèle dans SA décision. Ce n'est PAS la probabilité que la réponse soit juste.

0.28 (deux classes quasi à égalité)

Le LLM-judge déplace la logique de décision dans du parsing texte côté n8n ; JEV l'expose déjà calculée — et le doute devient un signal exploitable au lieu d'un angle mort.

Un LLM-judge vous donne un avis. JEV vous donne une classe, sa probabilité et son propre doute.

n8n - LLM - Jev Judge — D'où vient la confiance ?

Même réponse LLM, deux jeux de critères (mesure live)

Critères actuels (stricts)

choice : correct
P(correct) : 0.52
P(partiel) : 0.48
confidence : 0.28

Critères affinés (résumé accepté)

choice : correct
P(correct) : 0.76
P(partiel) : 0.24
confidence : 0.64

Bien définir le critère fait monter la confiance; c'est exactement la thèse JEV : on juge contre des critères explicites, pas à l'intuition.

Réponse juste mais courte → JEV hésitait entre « correct » et « partiel ». On précise le critère : il tranche.

Démo live — triage des messages clients - GoogleSheets

27 messages clients réels · JEV (TypeSafe) · 3 étiquettes · ~300 ms et $\approx 0,00002$ \$ par décision · run du 08/10/2026

Réf.: [Triage Msg Client - GoogleSheet](#)

14

urgent

Panne, incident critique,
sécurité, paiement bloqué
— action immédiate

10

à suivre

Demande ou question à traiter,
sans urgence immédiate

3

pas de suivi

Message informatif ou
remerciement — aucune action

Réponse = une étiquette + une probabilité (confiance). Sous le seuil 0,70, un humain reprend la main.

Message client (extrait)	Étiquette	Probabilité
Mon site est complètement en panne depuis 2 heures, personne ne peut commander.	urgent	1.00
Est-ce que vous proposez des réductions pour les étudiants ?	à suivre	0.86
Merci pour la livraison rapide, tout est parfait.	pas de suivi	1.00

Confiance observée : 0,72 – 1,00 (moyenne 0,96). La plus hésitante : le message « webinar » (0,72), à la frontière à suivre / pas de suivi.
JEV n'a pas de seed → de légères variations d'un run à l'autre ; la distribution, elle, reste stable.

Le modèle ne rédige rien : il lit sa distribution et décide — et sous le seuil, il passe la main.

Démo live — Carwash

27 messages clients réels · JEV (TypeSafe) · 3 étiquettes · ~300 ms et $\approx 0,00002$ \$ par décision · run du 08/10/2026

Réf.: [Drive Carwash](#)

JEV ne remplace pas la modélisation du problème. Le code établit ce que l'on sait avec certitude ; le state décrit correctement le monde ; JEV intervient là où il reste une décision sémantique bornée à prendre. Code = règles certaines. State = faits pertinents. JEV = jugement.

Code = règles certaines. State = faits pertinents. JEV = jugement.

Comment Jev choisit les bons emojis ?

Une phrase → des jugements → une sélection d'emojis pertinents

1 Tu saisis une phrase

L'utilisateur entre une phrase dans l'application.

« Je pars en vacances à la mer avec mon chien »



2 Un catalogue d'emojis

221 emojis prédéfinis. Chaque emoji devient une question indépendante pour Jev.



... (221 au total)

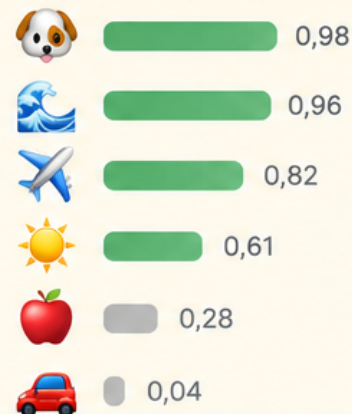
3 Jev évalue chaque emoji

Une seule requête à Jev avec 221 jugements oui/non. Jev retourne un score de pertinence (0 à 1) pour chaque emoji.

```
{
  model: "typesafe/jev-1.13",
  state: { request: "...", },
  questions: {
    "dog": {
      type: "nouil",
      instructions: "Does the emoji 'dog' fit request?"
    },
    ... 221 questions
  }
}
```

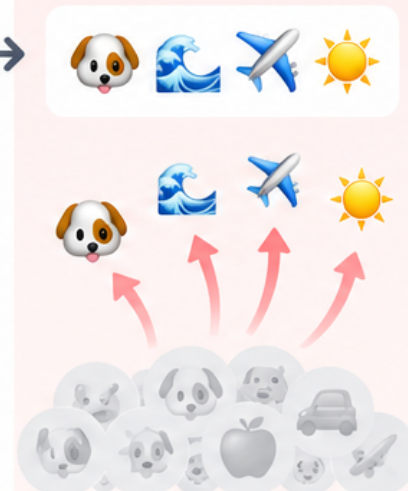
4 Filtrage et tri

On garde les emojis dont le score $\geq 0,50$, on les trie du plus pertinent au moins pertinent, et on en conserve au maximum 12.



5 Résultat affiché

Les meilleurs emojis sont sélectionnés et animés vers leur position avec Matter.js.



Rôle de chaque composant

Jev (typesafe/jev-1.13)	Évalue la pertinence sémantique de chaque emoji.
Code de l'application	Construit les questions, applique le seuil et trie les résultats.
Matter.js	Anime les emojis sélectionnés dans l'interface.



Principe clé

Jev ne génère pas les emojis : il examine 221 possibilités et attribue un score de pertinence à chacune, en fonction du sens de la phrase.



Mode secours

Si Jev n'est pas disponible, une recherche par mots-clés est utilisée (noms et étiquettes des emojis).

offline mode



Une phrase en langage naturel → des jugements de Jev → quelques emojis qui résument l'idée.

Meetup IA

Merci.

Des questions ?

Anne-Marie Roy · www.codeacademy123.com · 06 02 88 10 31